

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

Кафедра «Интеллектуальные информационные технологии»

ЛАБОРАТОРНАЯ РАБОТА №2
по дисциплине «Основы машинного обучения»
Тема: «Линейные модели для задач регрессии и
классификации»

Выполнил:
студент 2 курса
группы АС-67
Лискович И.Г.
Проверил:
Крощенко А. А.

Цель: Изучить применение линейной и логистической регрессии для решения практических задач. Научиться обучать модели, оценивать их качество с помощью соответствующих метрик и интерпретировать результаты.

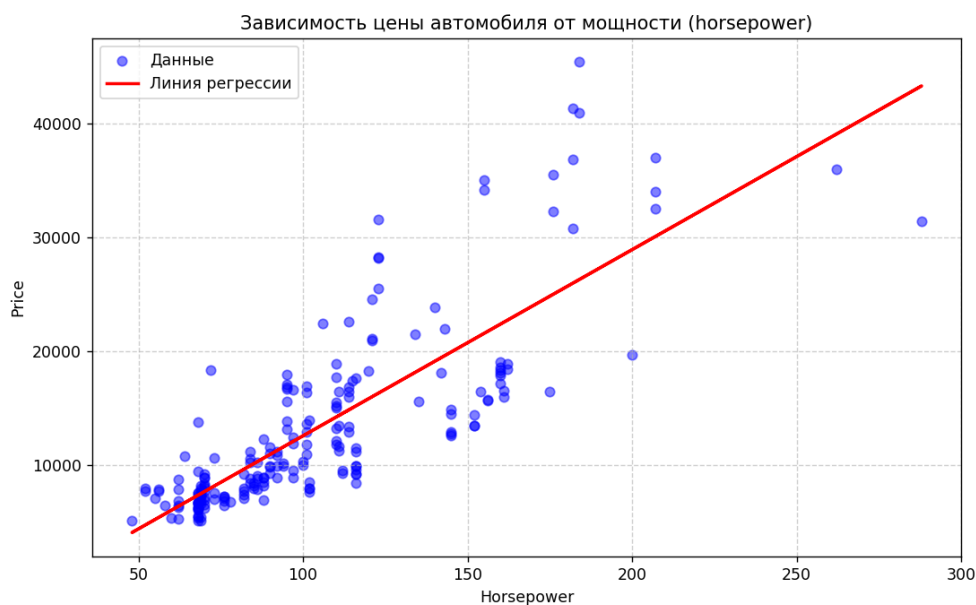
Общее задание: выполнить задания по варианту (регрессия и классификация), построить все требуемые визуализации и рассчитать метрики, написать отчет, создать пул-реквест в репозиторий с кодом решения и отчетом в формате pdf.

Используемые инструменты: Python, Pandas, Matplotlib, NumPy, Jupyter Notebook / Google Colab / PyCharm

Задания по вариантам:

Вариант 7

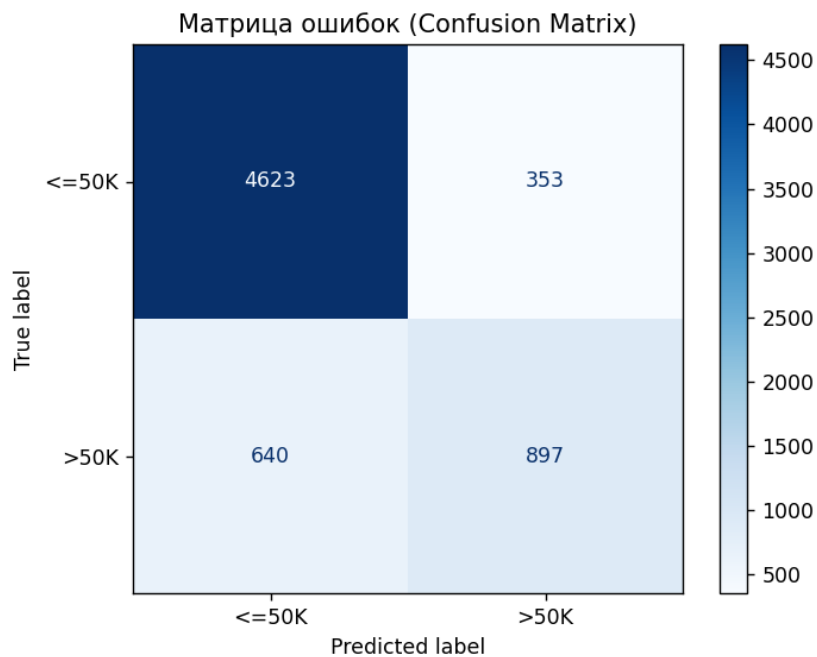
- Регрессия (Прогнозирование цены автомобиля)
 - Car Price Prediction
 - Предсказать цену автомобиля (price)
 - Задания:
 - загрузите данные. Выберите 5-6 числовых признаков (например, horsepower, citympg, enginesize);
 - обучите модель линейной регрессии;
 - рассчитайте R^2 и MAE;
 - визуализируйте зависимость price от horsepower с линией регрессии.



R^2 (коэффициент детерминации): 0.8160
MAE (средняя абсолютная ошибка): 2766.5951

- Классификация (Прогнозирование уровня дохода)
 - Adult Census Income
 - Предсказать, превышает ли доход \$50 тыс. в год (income >50K)
 - Задания:
 - загрузите данные, обработайте пропуски и категориальные признаки;
 - обучите модель логистической регрессии;

- рассчитайте Accuracy, Precision и Recall;
- постройте матрицу ошибок.



Accuracy: 0.8475
 Precision: 0.7176
 Recall: 0.5836

Код:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import mean_absolute_error, r2_score, accuracy_score, precision_score, recall_score,
confusion_matrix, ConfusionMatrixDisplay
from sklearn.preprocessing import StandardScaler

def regression_task():
    try:
        df = pd.read_csv('CarPrice_Assignment.csv')
    except FileNotFoundError:
        print("Ошибка: Файл 'CarPrice_Assignment.csv' не найден. Скачайте датасет и укажите правильный
        путь.")

    return

    features = ['horsepower', 'citympg', 'engine size', 'curbweight', 'highwaympg']
    target = 'price'

    missing_cols = [col for col in features + [target] if col not in df.columns]
```

```

if missing_cols:
    print(f"Внимание: В датасете отсутствуют колонки {missing_cols}. Проверьте названия столбцов.")
    return

df_reg = df[features + [target]].dropna()

X = df_reg[features]
y = df_reg[target]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)

print(f"R2 (коэффициент детерминации): {r2:.4f}")
print(f"MAE (средняя абсолютная ошибка): {mae:.4f}")

plt.figure(figsize=(10, 6))
plt.scatter(df_reg['horsepower'], df_reg['price'], alpha=0.5, color='blue', label='Данные')

z = np.polyfit(df_reg['horsepower'], df_reg['price'], 1)
p = np.poly1d(z)

plt.plot(df_reg['horsepower'], p(df_reg['horsepower']), color='red', linewidth=2, label='Линия
регрессии')

plt.title('Зависимость цены автомобиля от мощности (horsepower)')
plt.xlabel('Horsepower')
plt.ylabel('Price')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()

def classification_task():
    try:
        df = pd.read_csv('adult.csv')

```

```

except FileNotFoundError:

    print("Ошибка: Файл 'adult.csv' не найден. Скачайте датасет и укажите правильный путь.")

    return

df = df.replace(' ?', np.nan)

df = df.dropna()

target_col = 'income' if 'income' in df.columns else 'salary'

if target_col in df.columns:

    df[target_col] = df[target_col].str.strip()

    df['target'] = df[target_col].apply(lambda x: 1 if x in ['>50K', '>50K.'] else 0)

else:

    print("Ошибка: Не найдена целевая колонка 'income' или 'salary'.")

    return

X = df.drop([target_col, 'target'], axis=1)

y = df['target']

X = pd.get_dummies(X, drop_first=True)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000, random_state=42)

model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled)

accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(y_test, y_pred)

recall = recall_score(y_test, y_pred)

print(f"Accuracy: {accuracy:.4f}")

print(f"Precision: {precision:.4f}")

print(f"Recall: {recall:.4f}")

cm = confusion_matrix(y_test, y_pred)

```

```
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['<=50K', '>50K'])

disp.plot(cmap=plt.cm.Blues)

plt.title('Матрица ошибок (Confusion Matrix)')

plt.show()

if __name__ == "__main__":
    regression_task()
    classification_task()
```

Вывод: Я познакомился с